Documentação de engenharia reversa — Oficina AutoCerta

Escopo: análise estática dos arquivos entregues, leitura do esquema SQLite e execução de verificações sobre banco, código e criptografia. O pacote não contém documentação funcional prévia. O projeto é uma aplicação web Express com páginas HTML/JS estáticas e SQLite embutido via `node:sqlite`.

1. Visão geral e execução

• Runtime declarado: Node.js >= 22.13.0; o uso de `node:sqlite` requer uma versão compatível.

• Backend: Express 5, `express-session`, `bcryptjs`.

• Banco: SQLite em `db/oficina_db.sqlite`; `db/conexao.js` ativa `PRAGMA foreign_keys=ON` e, se o arquivo não existe, executa `db/oficina_db.sql`.

• Front-end: HTML/CSS/JavaScript sem framework; chamadas com `fetch` ao prefixo `/api`.

• Inicialização: `npm install`; `npm start`; abrir `http://localhost:3000`.

• Usuários de demonstração: `admin/admin123`, `ana.mecanica/ana123`, `carlos.recepcao/carlos123` (descritos no LEIA-ME e no HTML de login).

Fluxo macro: navegador → Express → middleware de sessão/rotas → consultas parametrizadas SQLite → JSON → atualização do DOM.

2. Requisitos funcionais identificados

ID | Requisito funcional | Evidência no código

RF01 | Redirecionar `/` para a tela de login. | `server.js`, rota `GET /`

RF02 | Autenticar usuário por login e senha. | `POST /api/login`, consulta `usuarios` e `bcrypt.compareSync`

RF03 | Consultar usuário da sessão atual. | `GET /api/me`

RF04 | Encerrar sessão. | `POST /api/logout` chama `req.session.destroy`

RF05 | Listar e pesquisar clientes por nome, CPF, telefone ou e-mail. | `GET /api/clientes?busca=...` e `public/js/clientes.js`

RF06 | Cadastrar cliente. | `POST /api/clientes`

RF07 | Editar cliente existente. | `PUT /api/clientes/:id`

RF08 | Listar veículos com dados do cliente associado. | `GET /api/veiculos`

RF09 | Cadastrar veículo vinculado a cliente. | `POST /api/veiculos`

RF10 | Listar agendamentos com dados de veículo e cliente. | `GET /api/agendamentos`

RF11 | Cadastrar agendamento para veículo existente. | `POST /api/agendamentos`

RF12 | Alterar a data de agendamento. | `PUT /api/agendamentos/:id`

RF13 | Mostrar mensagens de erro e redirecionar ao login se a sessão expirar. | `public/js/comum.js`, `api()`

RF14 | Formatar CPF e data para exibição e escapar texto inserido no HTML. | `public/js/comum.js`

Requisitos não identificados

Não há rotas de exclusão de clientes, veículos ou agendamentos. Não há edição de veículos nem alteração do serviço/veículo do agendamento. Embora existam três usuários com nomes de funções diferentes, o servidor não implementa autorização por perfil: qualquer usuário autenticado acessa as rotas protegidas. Não há gestão de usuários pela interface.

3. Regras de negócio

• **Autenticação:** login e senha são obrigatórios; credenciais inexistentes ou senha incorreta resultam em HTTP 401. Campos ausentes resultam em HTTP 400.

• **Sessão:** expiração nominal em 30 minutos; cookie `httpOnly`; middleware `exigirLogin` exige `req.session.usuario` nas rotas privadas.

• **Cliente:** nome obrigatório após `trim`; CPF normalizado para dígitos e deve conter 11 dígitos; e-mail, quando informado, deve corresponder ao padrão simples `não-espaço@não-espaço.domínio`; CPF duplicado é rejeitado; CPF é criptografado antes de persistir.

• **CPF duplicado:** a aplicação percorre os clientes e descriptografa CPFs para comparar o texto claro. O modelo de criptografia usa IV aleatório, logo o mesmo CPF não gera necessariamente o mesmo texto cifrado.

• **Veículo:** placa é normalizada removendo espaços/hífens e convertendo para maiúsculas. Regex aceita padrão antigo `ABC1234` e Mercosul `ABC1D23`. Modelo, tipo e `cliente_id` são obrigatórios; cliente deve existir; placa é única.

• **Agendamento:** veículo deve existir; data deve estar no formato `AAAA-MM-DD` e ser uma data de calendário real; serviço obrigatório. A edição atual só modifica a data.

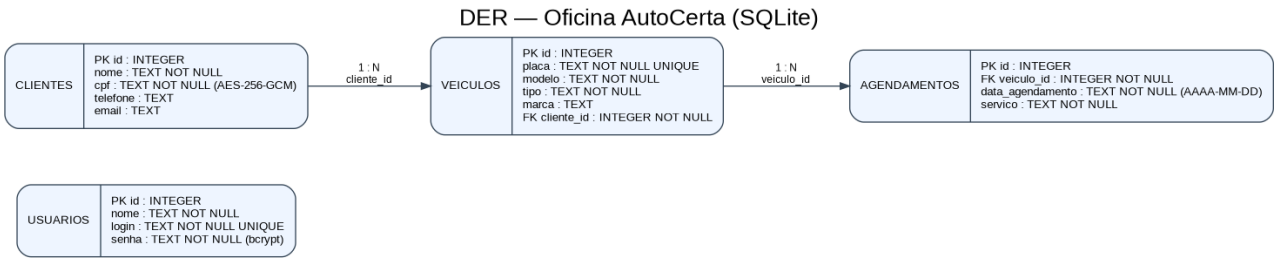
• **Relacionamentos:** um cliente pode ter vários veículos; um veículo pode ter vários agendamentos. Chaves estrangeiras são ativadas na conexão.

• **Ordenação:** clientes por nome; veículos por modelo; agendamentos por mês e data (via `strftime`).

• **Sem regra de conflito:** o código não impede dois agendamentos no mesmo dia/horário (não há campo de horário) e não impõe janela futura.

4. DER do banco

Modelo relacional identificado no esquema SQLite:



Arquivo visual: `der.png`; fonte reproduzível: `DER.dot`.

• `usuarios(id PK, nome, login UNIQUE, senha)` — senha armazenada como hash bcrypt.

• `clientes(id PK, nome, cpf, telefone, email)` — CPF criptografado com AES-256-GCM; não há restrição UNIQUE de CPF no SQLite, a duplicidade é verificada pela aplicação.

• `veiculos(id PK, placa UNIQUE, modelo, tipo, marca, cliente_id FK → clientes.id)`.

• `agendamentos(id PK, veiculo_id FK → veiculos.id, data_agendamento, servico)`.

Cardinalidades: `clientes 1:N veiculos`; `veiculos 1:N agendamentos`. `usuarios` é independente das entidades operacionais no esquema atual.

5. Mapa da API

Todas as respostas são JSON exceto arquivos estáticos/redirecionamento. Rotas privadas retornam `401 {"erro":"Sessão expirada. Faça login novamente."}` se não houver sessão. Mensagens abaixo são as encontradas no código; não há schema de validação externo.

Método e rota | Autenticação | Entrada | Sucesso | Validação/erros

`GET /` | Não | — | Redireciona para `/login.html` | —

`POST /api/login` | Não | JSON `{login, senha}` | `200 {ok:true}` e grava `{id,nome}` na sessão | `400` se faltar login/senha; `401` se login/senha inválidos

`POST /api/logout` | Não exigida explicitamente | — | `200 {ok:true}` após destruir sessão | Não diferencia falha de destruição; callback sempre responde sucesso

`GET /api/me` | Sim | — | `{id,nome}` | `401` sem sessão

`GET /api/clientes` | Sim | Query opcional `busca` | Array de clientes com CPF descriptografado | Busca case-insensitive por nome/e-mail, substring em telefone e dígitos do CPF

`POST /api/clientes` | Sim | `{nome,cpf,telefone?,email?}` | `201 {ok:true}` | `400` nome vazio, CPF diferente de 11 dígitos, e-mail inválido ou CPF duplicado

`PUT /api/clientes/:id` | Sim | Path `id`; corpo igual ao POST | `200 {ok:true}` | `404` cliente ausente; `400` mesmas validações do cadastro

`GET /api/veiculos` | Sim | — | Array com campos do veículo e `cliente_nome` | JOIN interno exclui veículos sem cliente válido

`POST /api/veiculos` | Sim | `{placa,modelo,tipo,marca?,cliente_id}` | `201 {ok:true}` | `400` placa inválida, modelo/tipo ausente, cliente ausente/inexistente ou placa duplicada

`GET /api/agendamentos` | Sim | — | Array com dados do agendamento, veículo e cliente; CPF descriptografado | Ordenado por mês/data

`POST /api/agendamentos` | Sim | `{veiculo_id,data_agendamento,servico}` | `201 {ok:true}` | `400` veículo inválido/ausente, data inválida ou serviço vazio

`PUT /api/agendamentos/:id` | Sim | Path `id`; `{data_agendamento}` | `200 {ok:true}` | `404` se agendamento não existe; `400` data inválida

Observações de contrato

• Os IDs são convertidos com `Number` em rotas de edição; não há validação explícita de inteiro positivo antes da consulta.

• As consultas usam placeholders SQL, evitando concatenação direta de valores do usuário nas consultas mostradas.

• A validação de CPF verifica apenas quantidade de dígitos, não os dígitos verificadores oficiais do CPF.

• `GET /api/clientes` retorna todos os CPFs descriptografados para qualquer usuário autenticado, o que amplia exposição de dado pessoal.

• A aplicação não define limite de tamanho para corpos, paginação, ordenação configurável ou rate limiting.

6. Mapa do front-end

Página | Script(s) | Comportamento | Rotas chamadas

`login.html` | `js/comum.js`, `js/login.js` | Login, mensagens de erro e aviso de sessão expirada | `POST /api/login`

`principal.html` | `js/comum.js` + `iniciarPagina()` inline | Painel com links para clientes, veículos e agendamentos; exibe nome e botão sair | `GET /api/me`, `POST /api/logout`

`clientes.html` | `js/comum.js`, `js/clientes.js` | Busca, tabela, modal de cadastro/edição | `GET /api/me`, `GET /api/clientes?busca=...`, `POST /api/clientes`, `PUT /api/clientes/:id`, `POST /api/logout` via sair

`veiculos.html` | `js/comum.js`, `js/veiculos.js` | Lista e cadastro de veículo; carrega clientes para seletor | `GET /api/me`, `GET /api/veiculos`, `GET /api/clientes`, `POST /api/veiculos`, `POST /api/logout`

`agendamentos.html` | `js/comum.js`, `js/agendamentos.js` | Lista agrupada por mês, cria agendamento e altera data | `GET /api/me`, `GET /api/agendamentos`, `GET /api/veiculos`, `POST /api/agendamentos`, `PUT /api/agendamentos/:id`, `POST /api/logout`

`css/style.css` | — | Estilo global, layout, tabela, modais, avisos | —

O helper `api()` envia JSON com `fetch`, tenta interpretar a resposta e lança erro quando `resp.ok` é falso. Para `401` em chamadas que não sejam login, redireciona para `/login.html?expirou=1`. `esc()` substitui caracteres HTML sensíveis antes de inserir dados em `innerHTML`.

7. Análise de segurança

Sessão

Pontos positivos: sessão no servidor via `express-session`; `saveUninitialized:false`; cookie `httpOnly`; expiração configurada para 30 minutos; sessão guarda apenas `id` e `nome`; rotas de dados usam middleware de autenticação; logout destrói a sessão.

Riscos: `secret` está fixo e previsível no código; não se configura `secure:true` (HTTPS), `sameSite` ou proxy confiável; o store padrão MemoryStore não é adequado a produção/múltiplas instâncias; não há regeneração de ID após autenticação explicitamente visível no código, aumentando risco de session fixation dependendo do fluxo; logout não exige login e sempre retorna sucesso; não há proteção CSRF para operações baseadas em cookie.

Hash de senha

O login compara a senha fornecida com o hash usando `bcrypt.compareSync`; `db/gerar_sql.js` usa `bcrypt.hashSync(s, 8)`, custo 8. Bcrypt é hash adaptativo com salt embutido, não criptografia reversível. Os registros SQL iniciais também armazenam hashes bcrypt. O custo 8 pode ser baixo para uma implantação atual e deve ser avaliado por benchmark.

Criptografia do CPF

`seguranca.js` usa AES-256-GCM, IV aleatório de 12 bytes e tag de autenticação; armazena `iv:tag:dados` em Base64. Isso é criptografia autenticada reversível. Entretanto, a chave é derivada de uma frase fixa e salt fixo dentro do código via `scryptSync`. Quem obtiver o código pode reconstruir a chave; não existe rotação ou gestão externa. CPFs são descriptografados para a API e para busca/duplicidade. A chave também é usada para os dados de exemplo, logo mudar a frase sem migrar/recriptografar dados inviabiliza a leitura dos CPFs existentes.

Outras observações

• Escaping do front-end reduz risco de XSS refletido/armazenado nas tabelas, desde que aplicado a todos os valores inseridos em HTML; preferir `textContent` e criação de nós.

• Não há autorização baseada em função, apesar dos nomes dos usuários sugerirem papéis distintos.

• Sem rate limiting, limite explícito de payload, logging de auditoria ou política de bloqueio de login.

• Dependências devem ser mantidas atualizadas e auditadas (`npm audit` em ambiente com rede).

Três melhorias prioritárias

• **Segredos e sessão:** mover segredo de sessão e chave de CPF para variáveis de ambiente/secret manager, com entropia forte e rotação documentada; configurar cookie `secure` em HTTPS, `sameSite:'lax'` ou política adequada, store persistente e `req.session.regenerate()` após login; implementar proteção CSRF para POST/PUT.

• **Autorização e exposição de dados:** adicionar papéis/escopos no servidor, limitar quais funções podem ler/editar cada recurso, retornar CPF mascarado por padrão e expor CPF completo só em fluxo autorizado; adicionar auditoria e rate limiting de login.

• **Validação e integridade:** validar dígitos verificadores do CPF; adicionar `UNIQUE`/índice de CPF por digest HMAC estável (sem usar ciphertext aleatório para comparação), mantendo a cifra separada; validar IDs inteiros, tamanho de campos e payload; testar regras via testes HTTP automatizados.

8. Testes e resultados

O script `docs/evidencias/verificacao.py` foi executado neste ambiente em 19/10/2026: **24 verificações passaram (24/24)** em 0,086 s. O resultado integral está em `docs/evidencias/resultado-testes.txt`.

Cobertura executada: tabelas e quantidade de registros de demonstração; unicidade de login e placa; chaves estrangeiras; formato das datas persistidas; relações sem órfãos; regex de placas; presença de middleware de sessão, consultas parametrizadas, bcrypt, cookie `httpOnly`, validações de CPF/data, logout; teste real de ida-e-volta AES-256-GCM com Node.js; escape HTML no front-end.

Limitação relevante: não foi possível concluir `npm install` devido a falha DNS ao acessar `registry.npmjs.org` (`EAI_AGAIN`). Assim, os testes acima são verificações de código/banco/criptografia, não execução real das rotas HTTP. Não declarar como aprovados testes de API que não foram executados. Depois de instalar dependências, executar manualmente os casos de API abaixo ou automatizá-los com Supertest.

Casos de teste para completar após instalação das dependências

ID | Caso | Resultado esperado

API01 | `GET /api/me` sem cookie | 401

API02 | Login sem senha | 400

API03 | Login com senha incorreta | 401

API04 | Login `admin/admin123` | 200, sessão criada

API05 | `GET /api/clientes` autenticado | 200, array

API06 | Busca por nome de cliente existente | cliente correspondente

API07 | Cadastro sem nome | 400

API08 | Cadastro com CPF com 10 dígitos | 400

API09 | Cadastro de CPF já existente | 400

API10 | Cadastro de cliente válido | 201

API11 | `PUT /api/clientes/999999` | 404

API12 | Placa inválida em `POST /api/veiculos` | 400

API13 | Placa já cadastrada | 400

API14 | Veículo com cliente inexistente | 400

API15 | Veículo válido associado a cliente válido | 201

API16 | Agendamento com data `2026-02-31` | 400

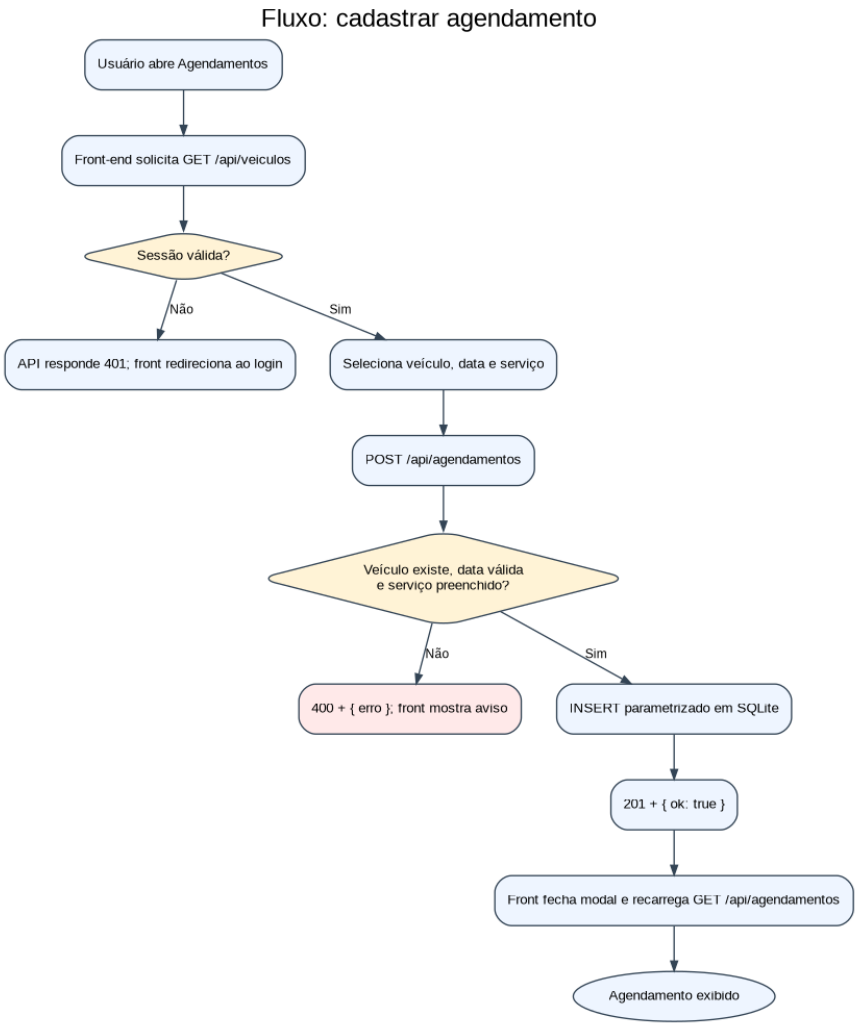
API17 | Agendamento sem serviço | 400

API18 | Agendamento com veículo inexistente | 400

API19 | Agendamento válido | 201

API20 | `PUT /api/agendamentos/999999` | 404

9. Fluxo documentado



O diagrama `fluxo-agendamento.png` representa o cadastro de agendamento: o front carrega veículos, a sessão é validada, usuário preenche veículo/data/serviço, a API valida os campos e existência do veículo, grava com consulta parametrizada e responde `201`; em erro responde `400` com mensagem para o front-end.

10. Código comentado

A cópia em `sistema_comentado/` inclui comentários nos arquivos centrais (`server.js`, `seguranca.js`, `db/conexao.js`, `db/gerar_sql.js` e scripts front-end) explicando intenção, segurança, dependências e limitações. Os comentários não alteram o comportamento funcional.